

SkillClaw

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

Let Skills Evolve Collectively with Agentic Evolver

arxiv.org/abs/2604.08377

ML Researchers & AI Systems Engineers



Agent Skills Remain Static After Deployment

Users Rediscover Solutions Independently — Every Time

■ STATIC SKILL ECOSYSTEM

- Skills installed manually from hub
- Never updated from runtime experience
- Failures not persisted beyond a session
- Each user re-discovers solutions independently
- No system-level knowledge accumulation

- Memory methods: tied to instances, hard to generalise
- Skill libraries: compressed but frozen — never updated

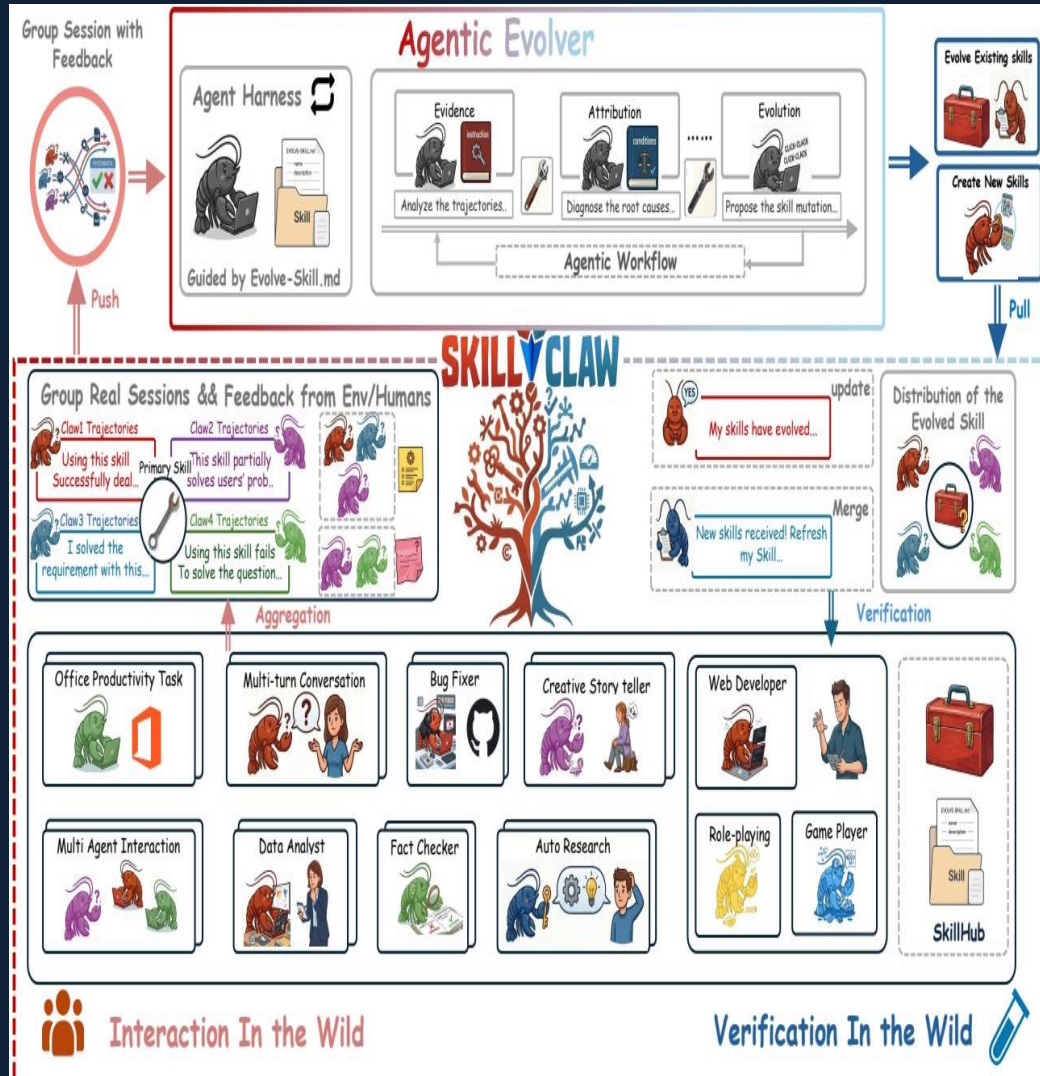
✓ EVOLVING SKILL ECOSYSTEM

- Skills improve automatically from usage
- Cross-user trajectories feed shared learning
- Failures & recoveries persist as evidence
- Single discovery benefits all users
- System-level knowledge accumulates over time

SkillClaw: closed-loop agentic evolution

Closed-Loop Pipeline: Multi-User Interaction → Shared Skill Evolution

Four-stage architecture for continuous, collective skill improvement



1 EVIDENCE

Aggregate session trajectories from all users by skill reference. Preserve full action–feedback chains.

2 ATTRIBUTION

Agentic evolver diagnoses root causes of failures and successes through open-ended reasoning.

3 EVOLUTION

Propose skill mutations — refine existing skills or create new ones from evidence.

4 DISTRIBUTION

Verify candidates; merge accepted skills into shared repository; synchronise back to all agents.

Agentic Evolver: Evidence → Attribution → Evolution

Three-stage pipeline driven by open-ended agent reasoning — not predefined rules

0 EVIDENCE

- 1 Analyze aggregated session trajectories
Grouped by referenced skill across all users
Identify behavioral patterns: success & failure modes
Full action–feedback causal chains preserved



0 ATTRIBUTION

- 2 Diagnose root causes of observed failures
Identify conditions under which skills break
Distinguish generalizable issues from idiosyncratic noise
Open-ended reasoning — no predefined diagnostic rules



0 EVOLUTION

- 3 Propose skill mutations based on attributed causes
Refine existing skill instructions or create new skills
Candidates verified before merging to shared repository
Conservative acceptance prevents regressions

The evolver is an autonomous agent performing open-ended reasoning over interaction evidence — not a rule-based system.

Cross-User Trajectory Aggregation Exposes Skill Behavioral Boundaries

Complementary signals from diverse contexts — impossible for a single user to generate

Compatible Claw-Style Systems

OpenClaw

CoPaw

IronClaw

PicoClaw

ZeroClaw

NanoClaw

NemoClaw

Why Multi-User Aggregation Matters

Diverse contexts → complementary signals

Different users exercise the same skill under varied conditions, revealing both when it works and when it breaks.

Generalizable vs. idiosyncratic

A single user rarely generates enough signal to separate a generalizable improvement from an idiosyncratic fix.

Skill behavioral boundaries

Aggregated failure modes map the boundary conditions of each skill, enabling targeted, evidence-based mutations.

Shared benefit

One user's discovery — surfaced through evidence — propagates to all users via the shared skill repository.

Case Study: Multimodal Creative Pipeline Skill — 6 Days of Evolution

Conservative verification: only 1 of 6 candidates accepted — preventing regressions

Day	Candidate Skill	Skill Function	Validator	Next-Day Action	1 / 6 Accepted Conservative gate prevents regressions
1	validate-tmp-workspace-inputs	Check /tmp_workspace inputs & environment setup	Accept	Upgrade to new best pool	
2	multimodal-input-validation-and-setup	Multimodal input validation & output env init	Reject	Keep current best pool	
3	multimodal-creative-task-pipeline	Multimodal creative pipeline (extract from PDF/video/image)	Reject	Keep current best pool	
4	multimodal-creative-task-pipeline (impr.)	Added image classification, visual generation, structured validation	Reject	Keep current best pool	
5	multimodal-creative-task-pipeline (impr.) + validate-required-input-files	Creative pipeline + per-file fail-fast validation	Reject	Keep current best pool	
6	multimodal-creative-task-pipeline (cand.)	Extended PDF-to-poster / document-to-visual paths	Reject	Not admitted to next cycle	

Case Study: Git Push Auth-Fallback Skill — Iterative Refinement Over 6 Days

3 of 6 days yielded accepted updates — successful iterative improvement before plateau

Day	Candidate Skill	Change Summary	Validator	Next-Day Action	3 / 6 Accepted Iterative refinement then natural plateau
1	git-push-with-auth-fallback	Safe fallback instead of blocking on push failure	Accept	Add to Safety best pool	
2	git-push-with-auth-fallback	Unified patch/bundle filenames, reduced inconsistency	Accept	Keep updated best pool	
3	git-push-with-auth-fallback + git-clone-to-directory	Auth-alternative paths & secrets audit; fixed subshell pitfalls	Accept	Keep current best pool	
4	(none)	Same-pool retest; validator confirmed current pool as best	Accept	Continue same best pool	
5	git-push-with-auth-fallback	Added "push hang treated as auth failure"; no improvement	Reject	Keep current best pool	
6	git-push-with-auth-fallback	Added identity config & filename consistency; no improvement	Reject	Not admitted to next cycle	

Key Takeaways: Three Properties That Distinguish SkillClaw

Implications for continuously improving LLM agent systems

01

Collective Evolution

Individual interactions contribute to a shared, continuously improving skill ecosystem. Knowledge discovered by one user propagates to all — preventing repeated rediscovery.

02

Fully Automatic

Skill evolution is driven entirely by runtime interaction. No manual curation, no explicit user intervention required. The system improves itself.

03

Agentic Evolution Paradigm

Skill updates emerge from open-ended agent reasoning over accumulated evidence — not from predefined update rules. Enables context-aware, evidence-driven mutations.

Implication → Deploy SkillClaw-style closed-loop evolution to build agent systems that compound experience across users and time — compatible with OpenClaw, CoPaw, IronClaw, PicoClaw, ZeroClaw, NanoClaw, NemoClaw. Evaluated on WildClawBench with Qwen3-Max backbone.